IMT Nord Europe
École Mines-Télécom
IMT-Université de Lille

CERI Systèmes Numériques – PPI
UV Internet des Objets

RAPPORT DE PROJET

TempHumiX

Un capteur de température et d'humidité connecté en MQTT, de la carte Arduino jusqu'à l'interface de supervision

```
PUBLISH  /iot/TEMPERATURE_48566  →  "24.4"
PUBLISH  /iot/HUMIDITY_48566     →  "41.3"
```


RÉALISÉ PAR Basma Dziri
POSTE 48566
ENCADRANTS JF Beguin · A Guilloteau · D Juge-Hubert · A Bouchama

DATE Novembre 2026
CODE SOURCE https://github.com/basmadziri03/IOT_BD

Sommaire

1 Introduction **3**

2 Architecture du système TempHumiX **3**

3 Niveau 1 – La couche objets **4**

3.1 Premiers circuits en simulation 4

3.2 Le capteur TempHumiX 6

3.3 Passage sur le boîtier réel Arduino UNO R4 WiFi 6

4 Niveau 2 – La couche réseau **7**

4.1 Connexion WiFi, adresse IP et heure NTP 7

4.2 Le projet dans le modèle OSI 8

4.3 Observation des trames avec Wireshark 9

4.4 Le routeur RUTX50 et la topologie 10

5 Niveau 3 – La couche application : MQTT **11**

5.1 Premiers tests avec MQTTX 11

5.2 Le client MQTT du boîtier 12

5.3 Qualité de service, messages retenus et jokers 14

6 Niveau 4 – Application et plateforme **17**

6.1 Clients MQTT en console et thread 17

6.2 Interface graphique PySide6 18

6.3 Stockage et visualisation 19

7 Cycle de développement et sécurité **20**

7.1 Développement 20

7.2 Sécurité par niveau 20

8 Difficultés et conclusion **21**

8.1 Difficultés rencontrées 21

8.2 Conclusion et perspectives 21

Récapitulatif des réponses aux questions **22**

1 Introduction

Cours Partie 1 – Introduction, pourquoi ? comment ? définition et concepts de l'IdO
Modèle OSI Vue d'ensemble, toutes les couches

Le cours définit un **objet connecté** comme un objet capable d'échanger des données avec d'autres entités physiques ou numériques, et résume l'Internet des Objets par la formule **IoT = M2M + Internet**. L'enjeu principal est de relier le **monde physique**, où l'on mesure une grandeur, au **monde numérique**, où l'on stocke, analyse puis agit en retour.

Le projet TempHumiX met en pratique toute cette chaîne. Un boîtier à base d'Arduino **mesure** la température et l'humidité, les **transmet** en WiFi avec le protocole MQTT, et une application sur PC les **affiche**, les **archive** et peut **renvoyer des commandes** au boîtier. Le projet s'est déroulé en trois étapes :

1. **Prise en main** dans le simulateur SimulIDE : LED, bargraphe, capteur DHT22, écran LCD, puis le circuit complet TempHumiX ;
2. **Communication MQTT** : passage sur la carte réelle Arduino UNO R4 WiFi, connexion au broker Mosquitto de l'école et analyse des trames avec Wireshark ;
3. **Supervision** : application Python avec interface PySide6, archivage SQLite et courbes.

On retrouve dans le projet les **contraintes** d'un objet IoT vues en cours : faible puissance de calcul, très peu d'IHM sur l'objet (un simple écran LCD) et un travail simple dont les résultats sont envoyés par le réseau. Mon poste porte le numéro **48566**, utilisé comme identifiant du capteur dans tous les topics MQTT. [1]

2 Architecture du système TempHumiX

Cours Partie 1 – Architecture d'un système IdO, modèle en niveaux
Modèle OSI Correspondance niveaux IoT / couches OSI (tableau 2)

Le cours présente l'architecture d'une solution IoT en niveaux : les **objets** (capteurs, actionneurs, unité de traitement), le **réseau** (transport des données, passerelles), puis la partie **application / plateforme** (stockage, analyse, services). Voici où se place chaque élément du projet :

Niveau	Rôle (d'après le cours)	Dans le projet TempHumiX
1 · Objets	Mesurer et agir sur le monde physique	Capteur DHT22, LED de vie, bargraphe, écran LCD, carte Arduino UNO R4 WiFi
2 · Réseau	Transporter les données, relier les réseaux	WiFi 2,4 GHz, routeur Teltonika RUTX50 (point d'accès, routeur, passerelle)
3 · Fog / Edge	Traiter les données près des objets	Broker Mosquitto sur le réseau local, serveur NTP de l'école
4 · Application	Stocker, visualiser, commander	IHM Python (PySide6), base SQLite, courbes matplotlib, client MQTTX

Table 1 – Les niveaux de l'architecture IoT appliqués au projet.

Les niveaux du cours ne se superposent pas exactement aux sept couches du modèle OSI. Le tableau suivant indique, pour chaque partie du rapport, les couches concernées.

1. Exemple : `/iot/TEMPERATURE_48566`. Les identifiants WiFi et MQTT sont dans des fichiers séparés, exclus du dépôt GitHub.

Partie du rapport	Ce qui est traité	Couches OSI
3 · Couche objets	Capteur, LEDs, LCD, microcontrôleur (liaisons locales : fil unique, bus I2C, GPIO)	hors OSI
4.1 · Connexion WiFi	Bande ISM 2,4 GHz, association au réseau, WPA2, adresse MAC	1 · 2
4.1 · Adresse IP, DHCP, ARP	Adresse 192.168.1.x, recherche du broker	3
4.1 · Mise à l'heure NTP	Requête UDP vers le serveur NTP	4 · 7
4.3 · Wireshark	Trame de 85 octets décomposée en couches	2 à 7
4.4 · Routeur RUTX50	Point d'accès, routeur (NAT), services	1 · 2 · 3 · 7
5 · MQTT (session)	CONNECT, Keep Alive, PINGREQ, DISCONNECT	5
5 · MQTT (format)	Valeurs en texte « 24.4 », pas de chiffrement TLS	6
5 · MQTT (messages)	Publication, abonnement, QoS, Retain, sur TCP 1883	4 · 7
6 · Plateforme	paho-mqtt (7) ; SQLite et courbes, locales au PC	7 · hors OSI
7 · Sécurité	WPA2 (2), TLS absent (6), authentification et ACL MQTT (7)	2 · 6 · 7

Table 2 – Couches du modèle OSI concernées par chaque partie du rapport.

À RETENIR

Toutes les données passent par le **broker** : le boîtier et l'IHM ne se parlent jamais directement. Ce point central se trouve sur le réseau local (Fog), et non dans un Cloud distant.

3 Niveau 1 – La couche objets

Cours Partie 1 – Niveau 1 : capteur, actionneur, unité de traitement, interface de communication ; développement de système embarqué
Modèle OSI Hors OSI : liaisons locales (fil unique, I2C, GPIO, USB série)

Le cours distingue le **capteur**, qui transforme une grandeur physique en signal électrique, l'**actionneur**, qui transforme l'énergie électrique en phénomène physique (lumière, mouvement, chaleur), l'**unité de traitement** (le microcontrôleur) et l'**interface de communication**. Il rappelle aussi que pour un système embarqué, le **matériel et le logiciel sont conçus en même temps**.

Cours	Boîtier
Capteur	DHT22 sur A0 (-40 à +80 °C, humidité)
Actionneurs	LED de vie (D2), bargraphe (D3 à D12), écran LCD
Traitement	UNO R3 en simulation, puis UNO R4 WiFi
Communication	Port série, puis module WiFi ESP32-S3

Table 3 – Éléments du niveau 1 dans le boîtier.

3.1 Premiers circuits en simulation

Chaque élément a d'abord été testé seul dans SimulIDE, avec une Arduino UNO R3, dans un circuit et un dossier séparés. Le premier circuit fait clignoter une LED sur D2 à travers une résistance de 100 Ω (500 ms allumée, 500 ms éteinte). Le bargraphe de 10 LEDs est relié de D3 à D12 à travers un réseau de résistances ; le programme fait un chenillard aller-retour avec une boucle `for`, une LED toutes les 100 ms.

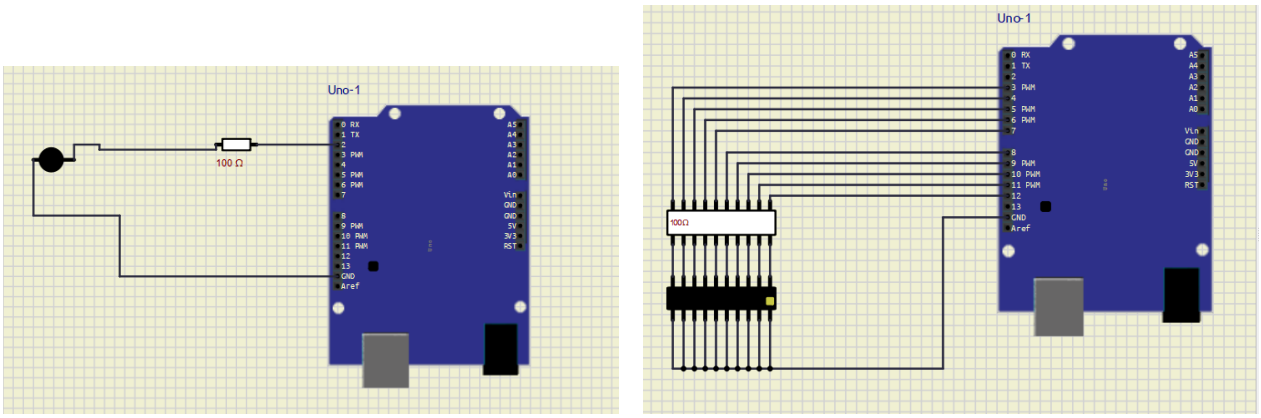


Figure 1 – Premier circuit : LED de vie sur D2 (à gauche) ; bargraphe de 10 LEDs relié de D3 à D12 (à droite).

Le capteur **DHT22** est relié à A0 et lu toutes les 2 s avec la librairie DHT d'Adafruit. Pour vérifier la lecture, une LED verte s'allume en dessous de 20 °C et une LED rouge au-dessus ; dans SimulIDE, on change la température avec les boutons + et –.

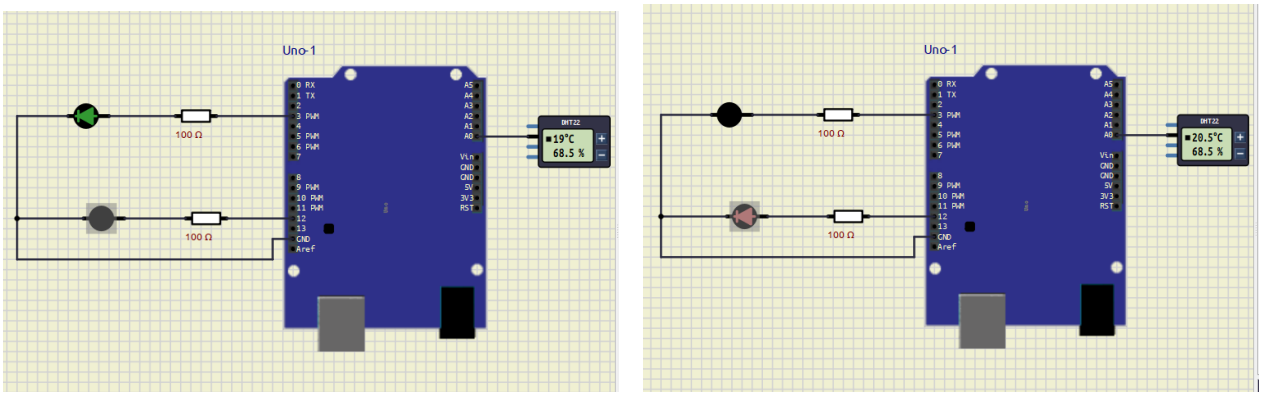


Figure 2 – DHT22 à 19 °C : LED verte allumée (à gauche) ; à 20,5 °C : LED rouge allumée (à droite).

Enfin, l'écran **LCD 16×2 en I2C** (contrôleur AIP31068) affiche d'abord « HELLO WORLD », puis « IMT NE – PPI » sur la première ligne et les mesures du DHT22 sur la deuxième, au format `T 22.5oC/H 68.5%`. [2]

> **Question 1 – TP partie 1, § 3.1.5**
> *Que pouvez-vous conclure par rapport aux différents circuits déjà réalisés ?*

RÉPONSE Chaque circuit valide **un seul bloc** du futur boîtier, avec son câblage et sa librairie : la LED de vie (un actionneur simple), le bargraphe (dix sorties pilotées en boucle), le DHT22 (le capteur, lu toutes les 2 s) et l'écran LCD en I2C (l'afficheur). En les testant séparément, on localise facilement les erreurs, comme la mauvaise librairie de l'écran. Il suffit ensuite d'**assembler des blocs qui fonctionnent déjà** pour obtenir le circuit complet TempHumiX. C'est la démarche de développement d'un système embarqué vue en cours : on avance par petites étapes en validant à chaque fois le matériel et le logiciel ensemble.

2. Attention : la bonne librairie est `LiquidCrystal_AIP31068_I2C.h`, et non `LiquidCrystal_I2C.h` comme dans la documentation.

3.2 Le capteur TempHumiX

Le circuit TempHumiX regroupe l'Arduino, le capteur, les LEDs (vie, Rx, Tx, Power), le bargraphe, le port série et l'écran LCD. Le programme fait clignoter la LED de vie à **2 Hz**, lit le DHT22 toutes les **2000 ms**, affiche la date et l'heure sur la première ligne du LCD et les mesures au dixième sur la deuxième, et envoie le tout sur le port série. Comme conseillé dans le TP, il est organisé en **machine d'état** cadencée par un `delay` dans la `loop`. La date venait d'abord d'un **dateur logiciel** qui fait avancer secondes, minutes, heures, jours, mois et années, avec des mois de 30 jours pour simplifier.



Figure 3 – Programme TempHumiX sur le boîtier réel : date du dateur logiciel (21/10/26), température et humidité.

3.3 Passage sur le boîtier réel Arduino UNO R4 WiFi

Sur le boîtier, l'**unité de traitement** change : l'UNO R4 (Arm Cortex-M4 à 48 MHz) est bien plus puissante que l'UNO R3 (ATmega328 à 16 MHz). Il a fallu :

- changer la librairie de l'écran pour `rgb_lcd.h` (Grove LCD RGB Backlight) ;
- remplacer le dateur logiciel par le **RTC intégré** de la carte ;
- organiser la `loop` en machine d'état de 500 ms.

Le programme est écrit et téléversé depuis l'Arduino IDE par le câble USB. Le module WiFi **ESP32-S3** de la carte devient l'**interface de communication** vers le niveau 2.

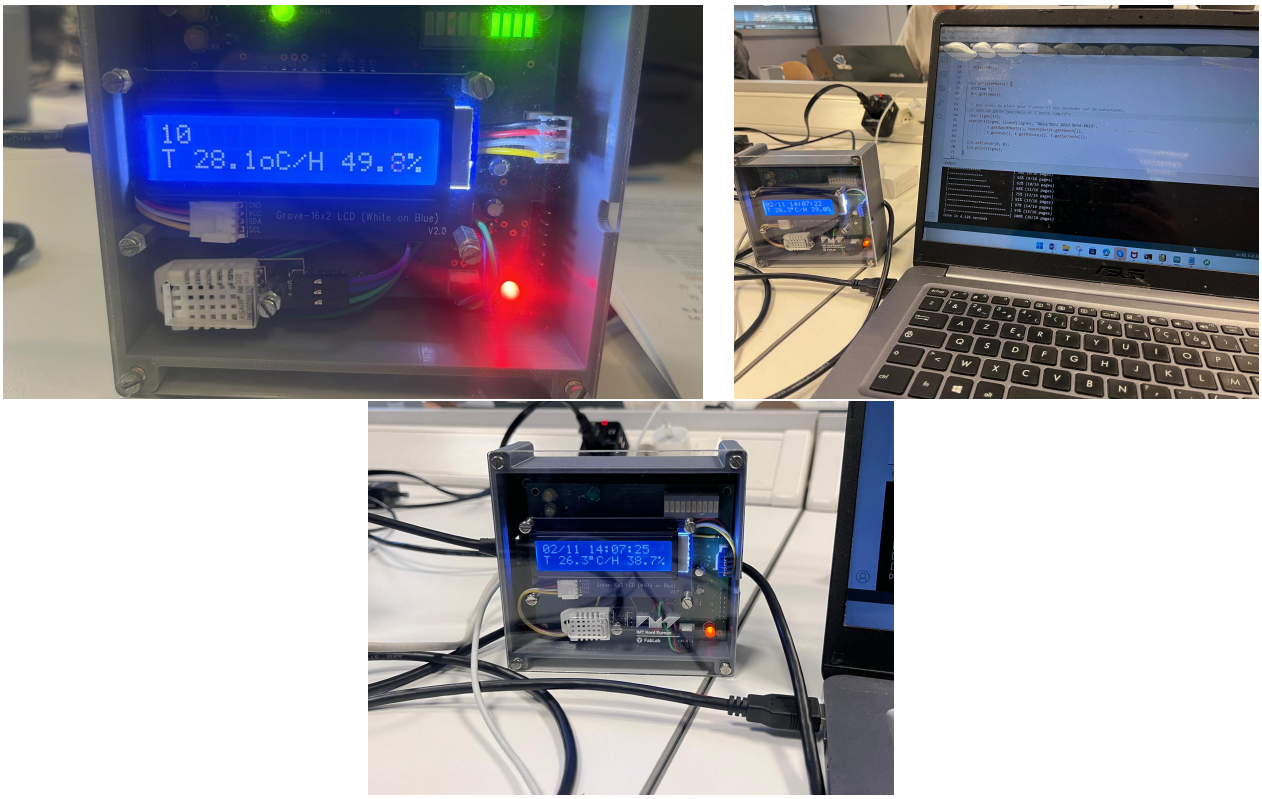


Figure 4 – Le boîtier réel : premiers essais du LCD et du bargraphe (à gauche) ; téléversement du programme depuis l'Arduino IDE (au centre) ; date et heure lues dans le RTC de l'UNO R4 (à droite).

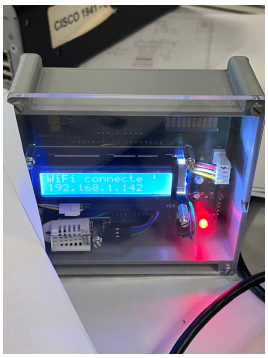
4 Niveau 2 – La couche réseau

Cours Partie 2 – Couche réseau : modèle OSI, codage, bandes de fréquences, portée, topologies, accès au médium, interconnexion
Modèle OSI Couches 1 à 4

4.1 Connexion WiFi, adresse IP et heure NTP

OSI couches 1 · 2 · 3

Le boîtier utilise le **WiFi**, un réseau à **moyenne portée** d'après le cours, dans la **bande ISM 2,4 GHz** (bande libre, comme Bluetooth ou Zigbee). La fonction `setup_wifi()` connecte la carte et renvoie `true` si elle obtient une adresse IP en moins de 10 s, `false` sinon. L'adresse est attribuée par le serveur **DHCP** du routeur.



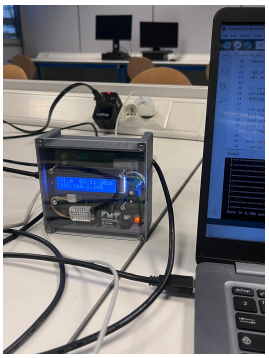
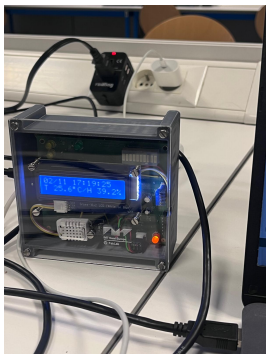



Figure 5 – « WiFi connecté » avec l'adresse obtenue par DHCP (à gauche) ; écran d'accueil avec la version `V1.0`, la date et l'adresse IP (au centre) ; affichage normal après la mise à l'heure par NTP (à droite).

L'écran d'accueil affiche ensuite la **version** du programme et l'**adresse IP**. La carte se met à l'heure auprès du **serveur NTP** de l'école (192.168.1.254, requête UDP), puis recopie l'heure dans le RTC. Si le serveur ne répond pas, l'écran affiche « Synchro NTP KO », mais la mesure continue sans être bloquée. [3]

Question 5 – TP MQTT, § 6.1.3

Modifier le code pour essayer de se connecter au réseau « IoT IMT Nord Europe 5G » : que se passe-t-il ? Pourquoi ?

RÉPONSE La connexion **échoue** : `setup_wifi()` renvoie `false` au bout de 10 s et l'écran affiche « Pas de WiFi ». Le module WiFi de l'UNO R4 (ESP32-S3) ne fonctionne que dans la bande **2,4 GHz** ; il ne peut ni voir ni rejoindre un réseau en 5 GHz. C'est un exemple du choix de bande de fréquence vu en cours : l'objet doit être compatible avec la bande utilisée par le réseau.

4.2 Le projet dans le modèle OSI

OSI couches 1 à 7

Question 13 – TP MQTT, question générale 1

Analysez le réseau IoT constitué jusqu'à présent (y compris le broker et le routeur RUTX50). Expliquez, en vous basant sur le modèle OSI (TCP/IP), l'acheminement de l'information dans le réseau avec les protocoles des différentes couches.

RÉPONSE Le tableau 4 donne le rôle de chaque équipement à chaque couche.

Couche	Boîtier TempHumiX	Routeur RUTX50	Broker / PC
7 Application	Client MQTT, client NTP	DHCP, DNS, interface web	MQTT (Mosquitto, paho-mqtt)
6 Présentation	Valeurs en texte « 24.4 »	–	Décodage, pas de chiffrement TLS
5 Session	CONNECT, Keep Alive	–	Sessions MQTT, PINGREQ
4 Transport	TCP 1883, UDP (NTP)	–	TCP, port 1883
3 Réseau	IP 192.168.1.x (DHCP)	Routage, NAT vers la 5G	IP 192.168.1.1 / .147
2 Liaison	WiFi 802.11, WPA2, MAC, ARP	Point d'accès / pont	Carte réseau (MAC)
1 Physique	Radio 2,4 GHz	Radio 2,4 / 5 GHz, Ethernet	Radio WiFi du PC

Table 4 – Rôle de chaque équipement du projet dans les sept couches du modèle OSI.

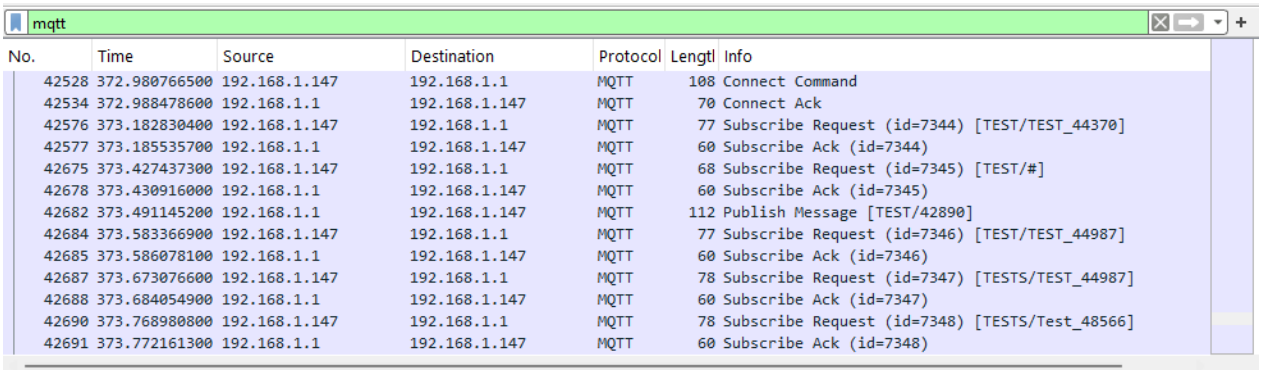
3. Mise à l'heure NTP : couche 4 (UDP) et couche 7 (protocole NTP).

Le boîtier obtient son adresse par **DHCP**, trouve l'adresse MAC du broker avec **ARP**, ouvre une connexion **TCP** sur le port 1883, puis envoie un **CONNECT** MQTT. Chaque mesure **descend les couches** dans le boîtier, et chaque couche ajoute son en-tête : c'est l'**encapsulation**. La trame traverse ensuite le routeur aux couches 1 à 3, puis **remonte les couches** jusqu'au broker, qui la redistribue aux abonnés par le chemin inverse. Le capteur DHT22 (fil unique), l'écran (bus I2C), la base SQLite et les courbes ne font pas partie du réseau.

4.3 Observation des trames avec Wireshark

OSI couches 2 à 7

J'ai capturé une session MQTTX complète avec Wireshark (filtre de capture `tcp port 1883`, puis filtre d'affichage `mqtt`). Le PC a l'adresse 192.168.1.147 et le broker 192.168.1.1.


mqtt

No.	Time	Source	Destination	Protocol	Lengtl	Info
42528	372.980766500	192.168.1.147	192.168.1.1	MQTT	108	Connect Command
42534	372.988478600	192.168.1.1	192.168.1.147	MQTT	70	Connect Ack
42576	373.182830400	192.168.1.147	192.168.1.1	MQTT	77	Subscribe Request (id=7344) [TEST/TEST_44370]
42577	373.185535700	192.168.1.1	192.168.1.147	MQTT	60	Subscribe Ack (id=7344)
42675	373.427437300	192.168.1.147	192.168.1.1	MQTT	68	Subscribe Request (id=7345) [TEST/#]
42678	373.430916000	192.168.1.1	192.168.1.147	MQTT	60	Subscribe Ack (id=7345)
42682	373.491145200	192.168.1.1	192.168.1.147	MQTT	112	Publish Message [TEST/42890]
42684	373.583366900	192.168.1.147	192.168.1.1	MQTT	77	Subscribe Request (id=7346) [TEST/TEST_44987]
42685	373.586078100	192.168.1.1	192.168.1.147	MQTT	60	Subscribe Ack (id=7346)
42687	373.673076600	192.168.1.147	192.168.1.1	MQTT	78	Subscribe Request (id=7347) [TESTS/TEST_44987]
42688	373.684054900	192.168.1.1	192.168.1.147	MQTT	60	Subscribe Ack (id=7347)
42690	373.768980800	192.168.1.147	192.168.1.1	MQTT	78	Subscribe Request (id=7348) [TESTS/Test_48566]
42691	373.772161300	192.168.1.1	192.168.1.147	MQTT	60	Subscribe Ack (id=7348)

Figure 6 – Début de la session dans Wireshark : connexion (CONNECT / CONNACK) puis abonnements (SUBSCRIBE / SUBACK).

Paquet	Sens	Champs relevés
CONNECT	PC → broker	Client ID `mqttx_0367f426`, Keep Alive 60 s, User Name et Password masqués ici (en clair dans la capture), MQTT v5.0
CONNACK	broker → PC	Reason Code 0 (Success) : connexion acceptée
SUBSCRIBE	PC → broker	Topic `TEST/TEST_48566`, QoS demandé 0
SUBACK	broker → PC	Granted QoS 0
PUBLISH	deux sens	Topic `/iot/TEMPERATURE_48566`, message `24.4`, QoS 0, Retain 0, DUP 0
PINGREQ / PINGRESP	PC ↔ broker	Envoyé 60 s après le dernier paquet du client (Keep Alive)
DISCONNECT	PC → broker	Reason Code 0 : fin volontaire de la session

Table 5 – Paquets MQTT relevés dans la capture Wireshark.

> **Question 6 – TP MQTT, § 6.2.1**
>
> *Où apparaît le mot de passe « • • • • • • • • • • » dans la capture ? Que concluez-vous sur la sécurité du port 1883 ?*

RÉPONSE Le mot de passe apparaît **en clair** dans le paquet **CONNECT** (trame 2723), champ *Password*. On le lit aussi directement dans la vue hexadécimale : `xx xx xx xx xx xx xx xx` (masqué dans cette version). Le port 1883 transporte MQTT **sans aucun chiffrement** : toute personne connectée au même réseau peut récupérer les identifiants et lire tous les messages. Pour sécuriser, il faudrait utiliser **MQTT sur TLS** (port 8883).

Question 7 – TP MQTT, § 6.2.1

Quelle est la taille (en octets) d'un paquet PUBLISH de température ? Comparez-la à celle d'une requête HTTP équivalente.

RÉPONSE La trame qui transporte `24.4` (trame 10961) fait **85 octets** :

```
[ Ethernet 14 o | IP 20 o | TCP 20 o | MQTT 31 o -> "24.4" ]
  couche 2 couche 3 couche 4 couches 5 a 7
```

Les 31 octets de MQTT comprennent l'en-tête fixe (2), la longueur du topic (2), le topic (22), les propriétés (1) et la valeur (4) : la donnée utile ne fait que **4 octets**. Une requête HTTP équivalente (`POST` avec les en-têtes Host, Content-Type, Content-Length…) fait facilement **200 à 400 octets**, plus la réponse du serveur, et demande souvent une nouvelle connexion TCP. MQTT est donc beaucoup plus léger, ce qui compte pour des objets sur batterie ou des réseaux lents.

Question 8 – TP MQTT, § 6.2.1

Que représente le champ « Keep Alive » ? Que se passe-t-il s'il n'y a aucun échange pendant cette durée ?

RÉPONSE Le Keep Alive est la **durée maximale sans échange** que le client annonce au broker dans le CONNECT, ici **60 s**. Si le client n'a rien envoyé pendant cette durée, il envoie un **PINGREQ** et le broker répond **PINGRESP**, pour montrer que la connexion est toujours vivante. Dans la capture, le ping part exactement 60 s après le dernier envoi (trames 17976 et 17978). Si le broker ne reçoit **rien pendant 1,5 × Keep Alive** (90 s), il considère le client comme perdu et ferme la connexion.

4.4 Le routeur RUTX50 et la topologie

OSI couches 1 · 2 · 3 · 7

Le cours distingue le **routeur**, qui relie deux réseaux de même nature, et la **passerelle**, qui relie deux réseaux de nature différente. Le RUTX50 joue les deux rôles. La **topologie** est en **étoile**, et l'accès au médium radio est **aléatoire** (CSMA).

Question 14 – TP MQTT, question générale 2

Quels services de couche applicative offre le RUTX50 ? À partir de la logique de l'architecture IoT, où situez-vous ces services (Cloud, Fog…) ? Quels rôles peut-il jouer pour l'interconnexion des réseaux (pont, passerelle…) ?

RÉPONSE Le RUTX50 (système RutOS) offre : serveur DHCP, DNS, serveur NTP, broker et client **MQTT**, passerelle **Modbus TCP/RTU ↔ MQTT**, envoi de données vers un serveur (« Data to Server » en HTTP ou MQTT), VPN (OpenVPN, IPsec, WireGuard), pare-feu, SNMP, SSH, interface web et gestion à distance (RMS). Ces services se situent au niveau **Fog / Edge** : le routeur est proche des objets et peut traiter les données localement avant de les envoyer vers le Cloud. Pour l'interconnexion, il joue le rôle de **pont** (WiFi et Ethernet dans le même réseau local), de **routeur** (NAT entre le réseau local et la 5G) et de **passerelle** (conversion de protocoles, par exemple Modbus → MQTT).

Question 15 – TP MQTT, question générale 3

Est-ce que le RUTX50 est équipé d'un MCU ou d'un MPU ? Quel système utilise-t-il ?

RÉPONSE C'est un **MPU** : un processeur Qualcomm IPQ4018 (ARM Cortex-A7 quadricœur à 717 MHz) avec 256 Mo de RAM et 256 Mo de flash externes. Il utilise **RutOS**, un système basé sur **Linux / OpenWrt**. Un MPU est nécessaire pour faire tourner en même temps de nombreux services ; un MCU comme l'Arduino n'exécute qu'un seul programme, sans système d'exploitation.

Question 16 – TP MQTT, question générale 4

Est-ce que les connexions WiFi sur le RUTX50 sont illimitées ? Pourquoi ? Quels risques pour la sécurité ? Quelle est la topologie ?

RÉPONSE Non : environ **150 clients** au maximum. La limite vient du temps d'antenne partagé (plus il y a de clients, moins chacun a de débit, et l'accès aléatoire provoque des collisions), de la mémoire du routeur et de la plage DHCP. Les **risques** : le mot de passe WiFi est partagé et écrit dans le sujet, donc n'importe qui peut rejoindre le réseau ; le broker n'est pas chiffré et le compte MQTT est commun, donc un intrus peut **lire les mesures** ou **envoyer des commandes** aux boîtiers des autres ; il peut aussi saturer le réseau (déni de service) ou créer un faux point d'accès. La topologie est en **étoile** : tous les clients passent par le point d'accès central.

5 Niveau 3 – La couche application : MQTT

Cours Partie 2 – Niveau 3 : protocoles de la couche application (REST, CoAP, MQTT, XMPP, AMQP, DDS) ; types de trafic
Modèle OSI Couches 5 · 6 · 7 sur TCP (couche 4)

Parmi les protocoles présentés en cours, **MQTT** est un protocole de **publication / abonnement** au-dessus de TCP/IP. C'est un trafic de type **producteur / consommateur** : le boîtier **publie** une valeur sur un **topic**, le **broker** la reçoit et la redistribue à tous les **abonnés**. Contrairement à REST ou HTTP, de type client / serveur, l'abonné n'a pas besoin de demander la donnée : il la reçoit dès qu'elle est publiée.

5.1 Premiers tests avec MQTTX

OSI couches 5 · 7

J'ai d'abord testé le broker avec le client MQTTX : connexion avec les paramètres du TP, publication d'un message JSON sur un topic de test, puis abonnement au même topic.

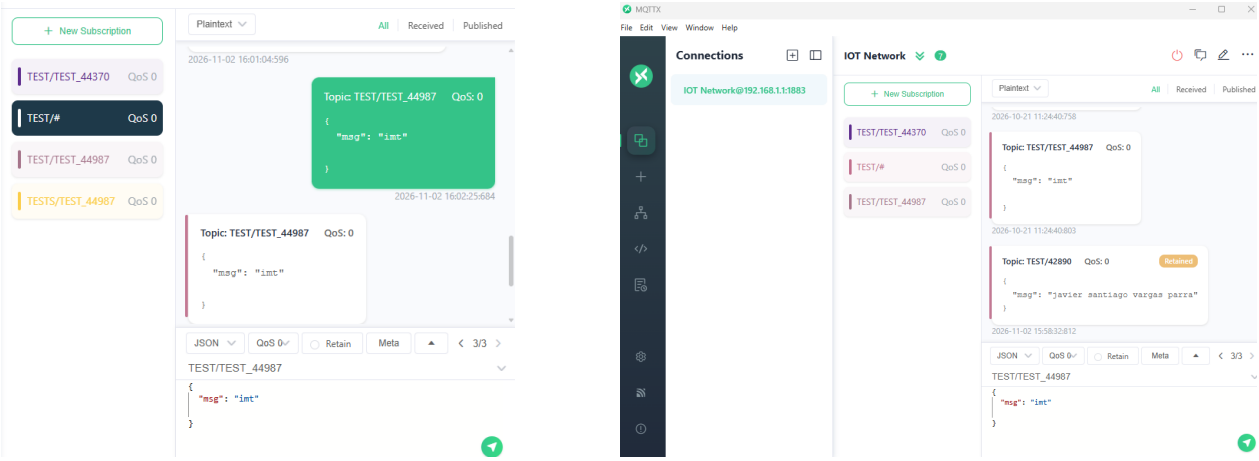


Figure 7 – Publication et réception d'un message de test dans MQTTX (à gauche) ; message retenu reçu dès l'abonnement (à droite).

Questions 2 à 4 – TP MQTT, § 5.2.4 « Jouons avec MQTT »

Essayez de souscrire à une variable d'un autre poste. Essayez de souscrire en utilisant le topic « TESTS/# ». Essayez de publier une variable en cochant le bouton Retain. Pour chacun des essais, donnez une explication et dites ce que vous remarquez.

RÉPONSE

- **Variable d'un autre poste** (question 2) : on reçoit sans problème les messages de l'autre poste. Le broker **n'isole pas les utilisateurs** : tout le monde se connecte avec le même compte et les droits d'accès (ACL) autorisent tous les topics de test. N'importe qui peut donc lire, et même publier, sur le topic d'un autre.
- **Topic `TESTS/#`** (question 3) : on reçoit les messages de **tous les postes** de la salle. Le joker `#` remplace tous les niveaux qui suivent `TESTS/` ; certains messages arrivent immédiatement avec l'étiquette *Retained*.
- **Publication avec Retain** (question 4) : le broker **garde le dernier message** du topic. Tout client qui s'abonne ensuite le reçoit immédiatement, même s'il était absent au moment de la publication. C'est pratique pour connaître tout de suite le dernier état d'un capteur.

5.2 Le client MQTT du boîtier

OSI couches 4 · 5 · 7

La fonction `setup_mqtt()` connecte la carte au broker avec l'identifiant `IdO_Client_48566` et renvoie 0, ou le code d'erreur MQTT en cas d'échec. La carte joue les deux rôles du modèle :

Rôle	Topics	Contenu
Producteur (publie)	`VERSION`, `TEMPERATURE`, `HUMIDITY`, `GET_DATE`	Version X.Y, mesures au dixième, date JJ/MM/AAAA HH :MM :SS
Consommateur (s'abonne)	`COMMAND`	10 = synchro NTP, 20 / 30 = démarrer / arrêter la mesure, 40 / 50 / 60 = clignotement lent / rapide / personnalisé
Consommateur (s'abonne)	`BARGRAPH`, `BLINK`	Valeur de 0 à 1023 ; période multiple de 100 ms

Table 6 – Topics publiés et souscrits par le boîtier (suffixe _48566, préfixe /iot/).

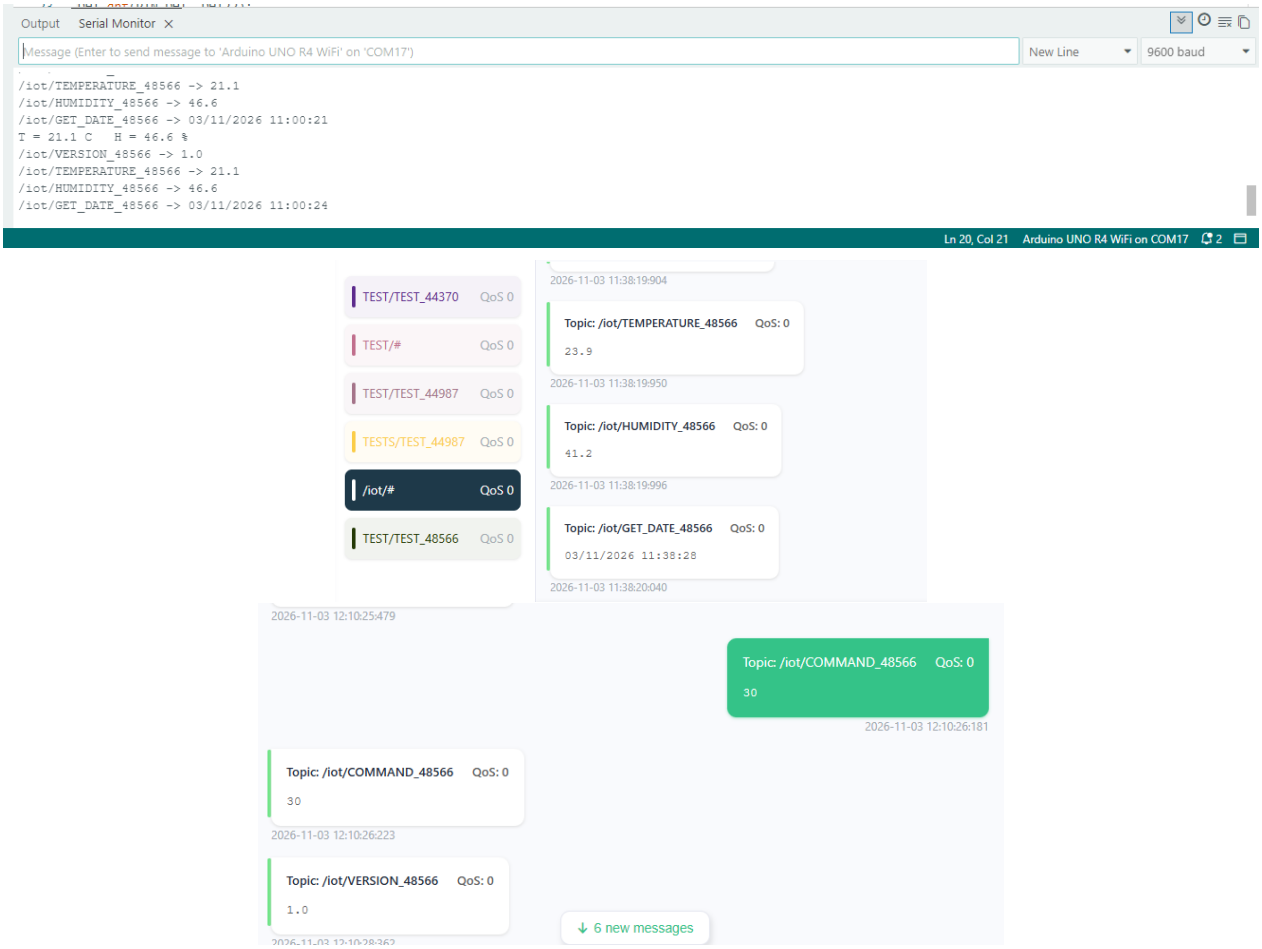


Figure 8 – Moniteur série : publications de la carte (en haut) ; mesures reçues dans MQTTX avec l'abonnement `/iot/#` (en bas à gauche) ; commande 30, arrêt des mesures, envoyée depuis MQTTX (en bas à droite).



Figure 9 – Le boîtier affiche « Mesure arrêtée » après la commande 30.

La fonction `onMqttMessage()` lit le topic et le message, convertit le message en entier, puis applique la commande. La période du clignotement personnalisé doit être un multiple de 100 ms ; la `loop` tourne donc toutes les 50 ms. La commande 10 est traitée dans la `loop` plutôt que dans le callback MQTT, car la synchro NTP peut durer jusqu'à 1 s.

J'ai ensuite amélioré la communication : la carte ne publie la température, l'humidité et la date **que si une valeur a changé**, en comparant les valeurs au dixième, et la version seulement après la commande 10.

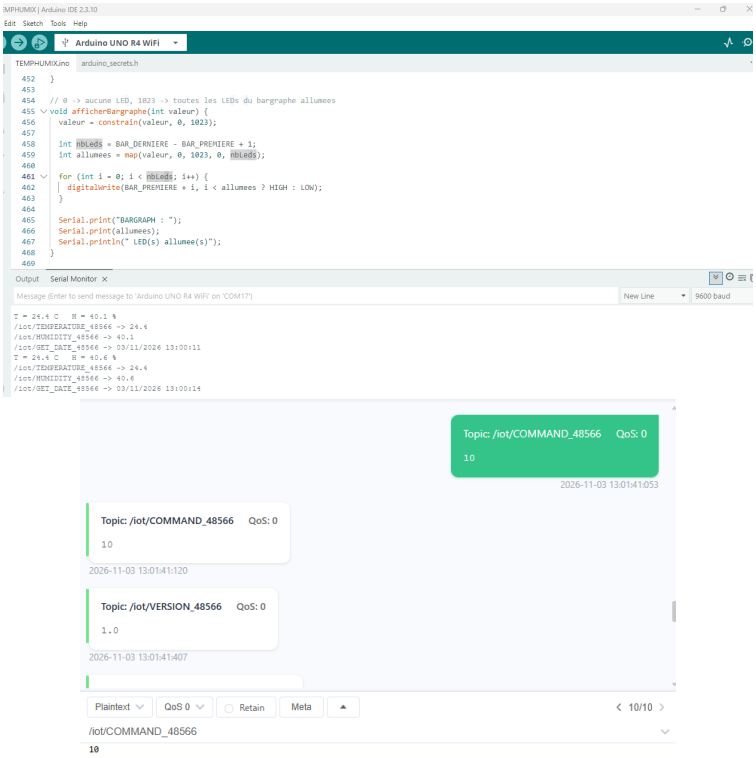


Figure 10 – Publication seulement quand une valeur change (à gauche) ; commande 10 : la carte se met à l'heure puis publie sa version (à droite).

À RETENIR

MQTT est **léger** (en-tête fixe de 2 octets, 31 octets pour une mesure) et **événementiel** : on ne publie que lorsqu'il y a une nouvelle information. Moins de messages, c'est moins de trafic et moins d'énergie consommée par le boîtier.

5.3 Qualité de service, messages retenus et jokers

OSI couche 7 (et TCP, couche 4)

Questions 9 et 10 – TP MQTT, § 6.2.2

Publier la variable de test avec l'option Retain, puis souscrire après la publication : que reçoit votre client ? Publier ensuite un message vide avec Retain : que devient le message retenu ?

RÉPONSE **Question 9** : le client reçoit **immédiatement** le message `Bonjour retain`, avec l'étiquette *Retained*, alors qu'il a été publié **avant** l'abonnement : le broker l'avait gardé en mémoire. **Question 10** : le message retenu est **effacé** par le broker ; un nouveau client qui s'abonne au topic ne reçoit plus rien.

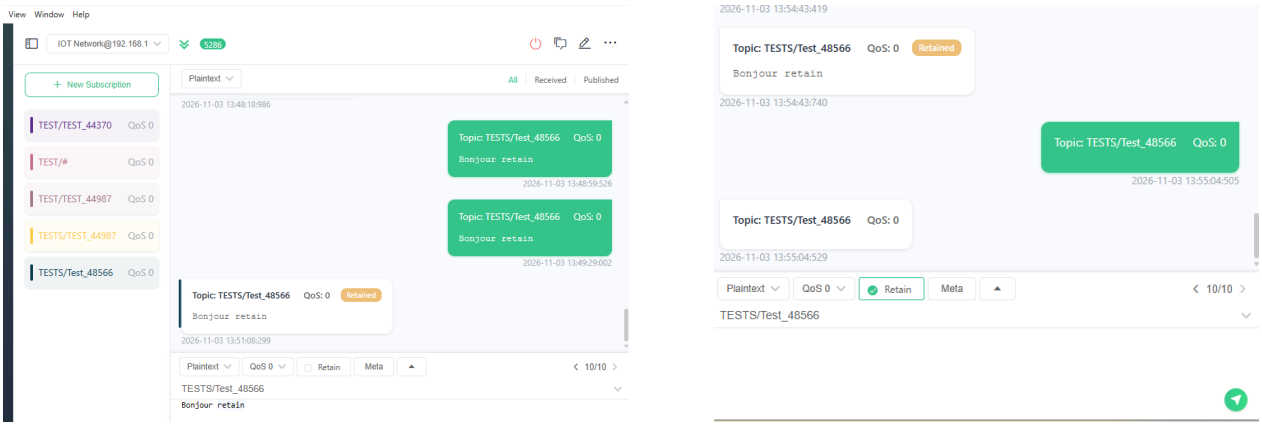


Figure 11 – Message retenu reçu à l'abonnement (à gauche) ; message vide publié avec Retain pour l'effacer (à droite).

Question 11 – TP MQTT, § 6.2.2

Publier avec QoS 0, 1 puis 2 et observer les paquets échangés dans Wireshark.

RÉPONSE

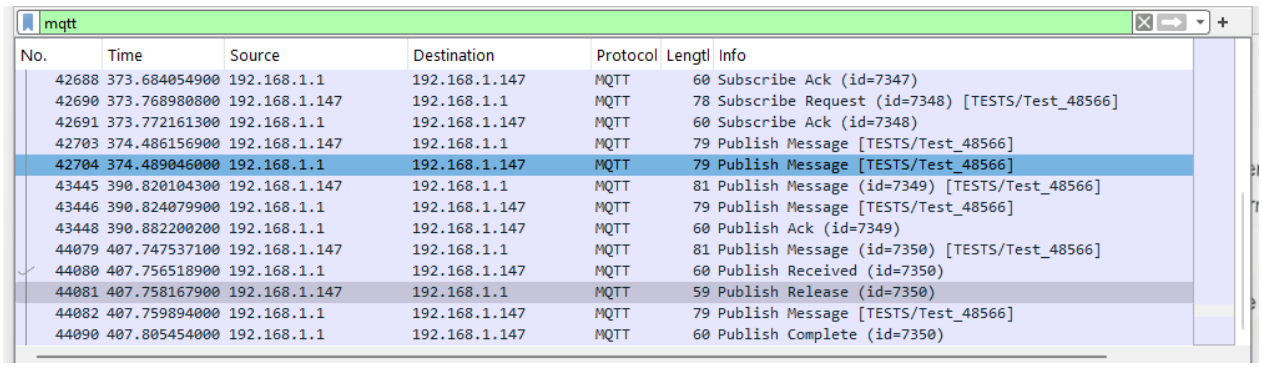


Figure 12 – Échanges pour QoS 0, 1 et 2 dans Wireshark.

QoS	Paquets observés
0	PUBLISH seul
1	PUBLISH → PUBACK (id 7349)
2	PUBLISH → PUBREC → PUBREL → PUBCOMP (id 7350)

Table 7 – Paquets échangés selon le QoS.

En **QoS 0**, le message est envoyé une seule fois, sans accusé de réception (« au plus une fois »). En **QoS 1**, le broker confirme par un PUBACK portant le même identifiant (« au moins une fois », doublons possibles). En **QoS 2**, l'échange en quatre paquets garantit une livraison « exactement une fois ». Les PUBLISH en QoS 1 et 2 font **2 octets de plus** (81 au lieu de 79) : c'est l'identifiant du message. Les copies reçues en retour arrivaient en QoS 0, car l'abonnement était en QoS 0 : le broker utilise le **plus petit** QoS entre la publication et l'abonnement. Avec un abonnement en QoS 2, le message est bien reçu en QoS 2.

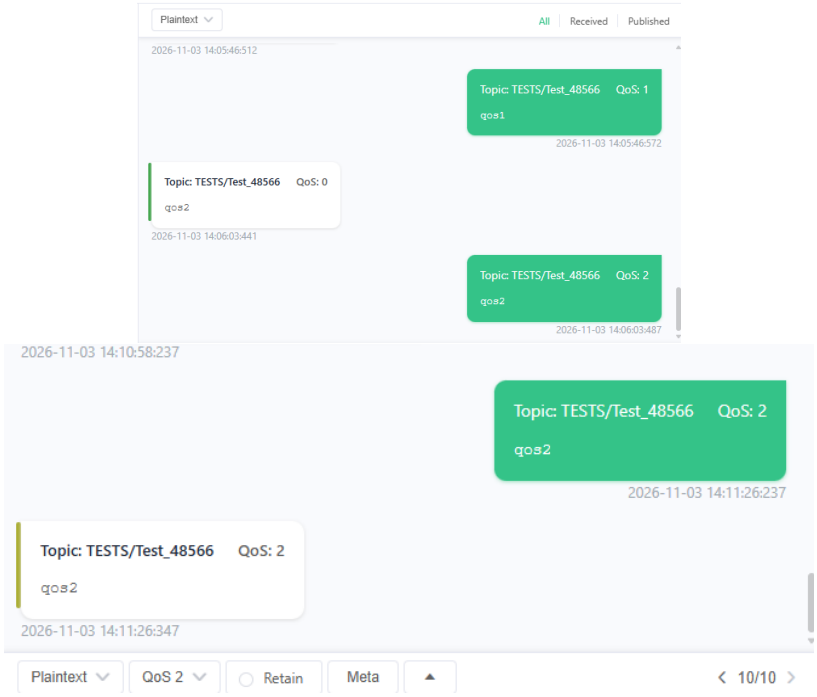


Figure 13 – Messages envoyés en QoS 1 et 2 mais reçus en QoS 0 (à gauche) ; avec un abonnement en QoS 2, réception en QoS 2 (à droite).

Question 12 – TP MQTT, § 6.2.2

Souscrire successivement à TESTS/+, TESTS/# puis $SYS/# (clients connectés, messages reçus…).

RÉPONSE

- `TESTS/+` : le joker `+` remplace **un seul niveau**. On reçoit `TESTS/Test_48566` et les tests des autres postes, mais pas un topic à deux niveaux comme `TESTS/a/b`.
- `TESTS/#` : le joker `#` remplace **tous les niveaux** suivants et doit être placé en dernier ; on reçoit en plus tous les sous-niveaux.
- `$SYS/#` : on reçoit les **statistiques du broker**, par exemple 214 811 messages reçus et environ 1 Mo envoyé, ainsi que les clients connectés et la version. Les topics commençant par `$` ne sont pas couverts par un simple `#`.

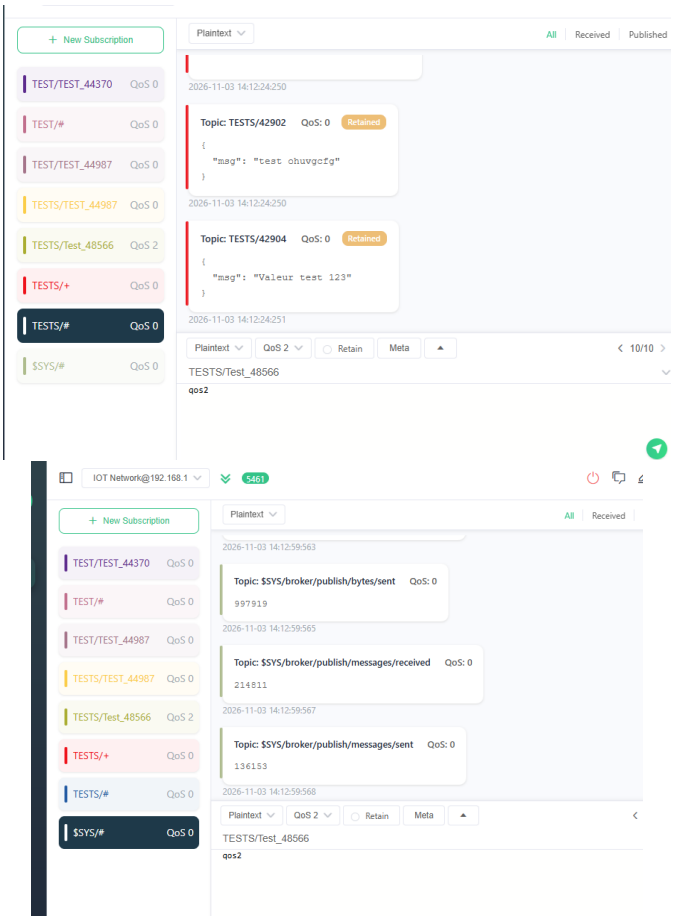


Figure 14 – Abonnement à `TESTS/+` : messages retenus des autres postes (à gauche) ; abonnement à `$SYS/#` : statistiques du broker (à droite).

6 Niveau 4 – Application et plateforme

Cours Partie 1 – Niveaux 3 et 4 : Cloud, Fog, Edge ; plateformes IoT et leurs composants
Modèle OSI Couche 7 (paho-mqtt) ; SQLite et courbes hors OSI

D'après le cours, une **plateforme IoT** apporte la connectivité, la gestion des appareils, le **stockage** et la **visualisation** des données. Dans le projet, cette partie est réalisée localement par une application Python, ce qui correspond plutôt à une architecture **Fog / Edge** qu'à un vrai Cloud.

6.1 Clients MQTT en console et thread

J'ai écrit une classe `ClientMQTT` avec la librairie `paho-mqtt`, puis deux programmes console : un client en **lecture** qui affiche les mesures reçues, et un client en **écriture** avec un menu pour envoyer les commandes. Ils tournent en même temps grâce à deux identifiants différents (`Client_48566` et `Client_E48566`).

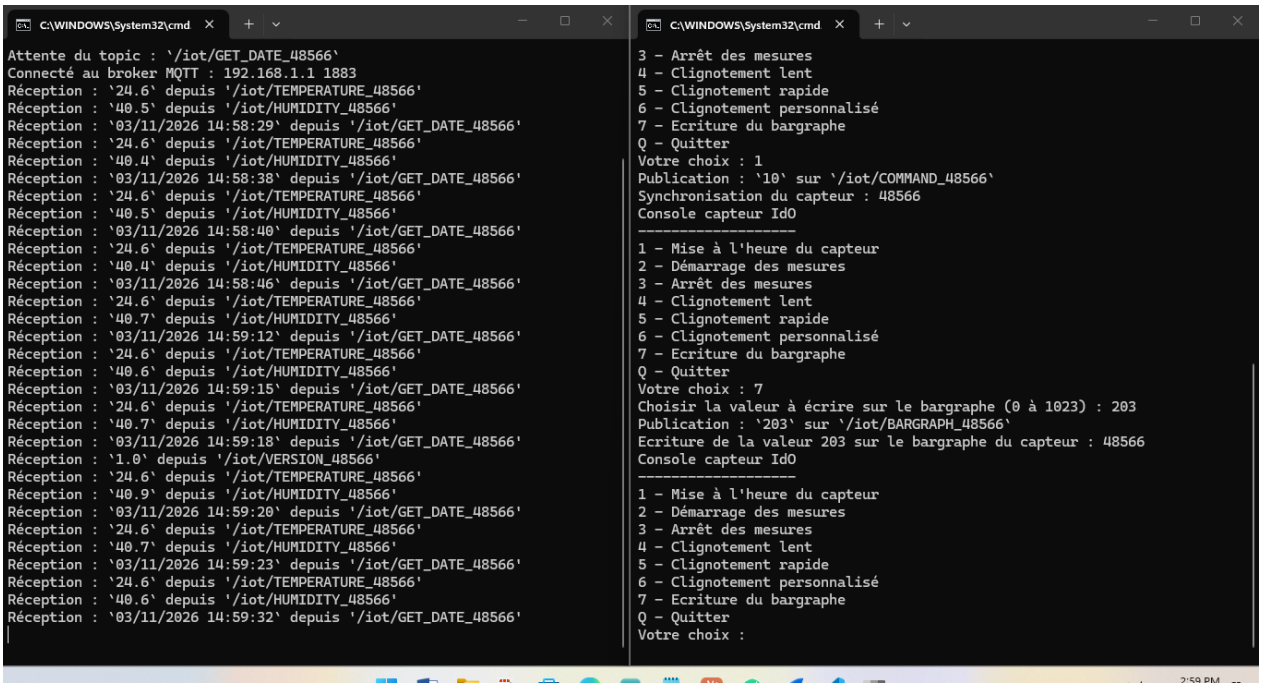


Figure 15 – Lecture (à gauche) et écriture (à droite) en parallèle : le choix 1 envoie la commande 10, la carte publie sa version et le programme de lecture la reçoit.

Une classe `ThreadMQTT` écoute ensuite le broker **en arrière-plan**, par tranches d'une seconde, jusqu'à ce qu'on l'arrête. `ClientMQTT` mémorise les dernières valeurs reçues : le programme reste disponible pendant la réception, ce qui est indispensable pour une interface graphique.

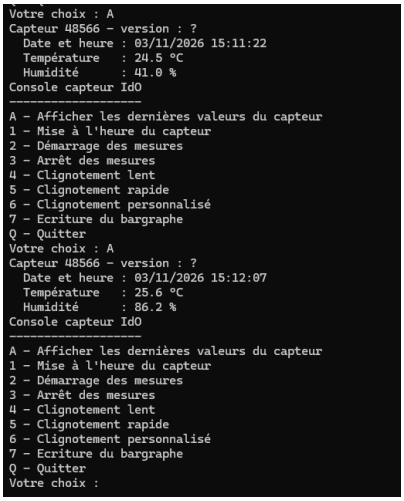


Figure 16 – Choix A : valeurs mises à jour en arrière-plan par le thread.

6.2 Interface graphique PySide6

La fenêtre a été dessinée dans **Qt Designer**, puis convertie en Python avec `pyside6-uic`. Chaque bouton est relié à une méthode (`connectMQTT`, `synchroDateTime`, `stopMeasure`…) et un **QTimer** met à jour l'affichage toutes les secondes.

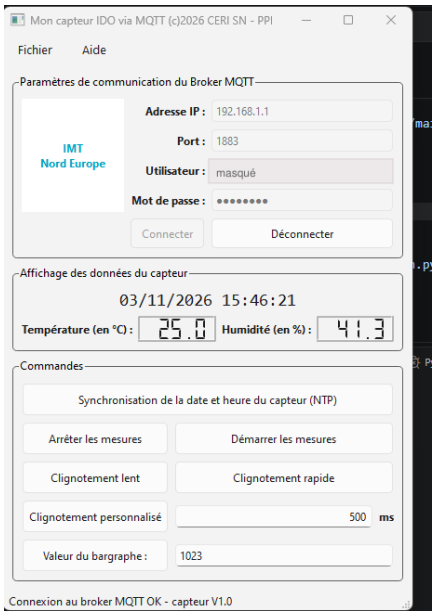


Figure 17 – L'interface connectée au boîtier.

Notice de fonctionnement

1. Connecter le PC au WiFi « IoT IMT Nord Europe » et lancer `ihm_mqtt_simple/main.py`.
2. Vérifier les paramètres du broker, puis cliquer sur **Connecter** ; les commandes restent grisées avant la connexion.
3. La date, la température et l'humidité se mettent à jour toutes les secondes. La barre d'état indique la connexion, la version du capteur et le nombre de mesures archivées.
4. Utiliser les boutons : synchronisation NTP, arrêt ou démarrage des mesures, clignotement lent, rapide ou personnalisé (période en ms), bargraphe (valeur de 0 à 1023 puis Entrée).
5. **Afficher les courbes archivées** ouvre la fenêtre des graphiques ; **Déconnecter** arrête la réception.

Les saisies sont contrôlées : une adresse IP invalide, une période hors limites ou une valeur de bargraphe supérieure à 1023 sont refusées avec un message.

6.3 Stockage et visualisation

Chaque mesure est enregistrée dans la base **SQLite** `IOT.db` (table `IOT_DATA`) à la réception de `GET_DATE`, le dernier topic publié par la carte. La base est ouverte dans le thread MQTT, car une connexion SQLite ne s'utilise que dans le thread qui l'a créée. La date est convertie au format AAAA-MM-JJ, sinon les requêtes `BETWEEN` et `ORDER BY` ne trieraient pas les dates correctement.

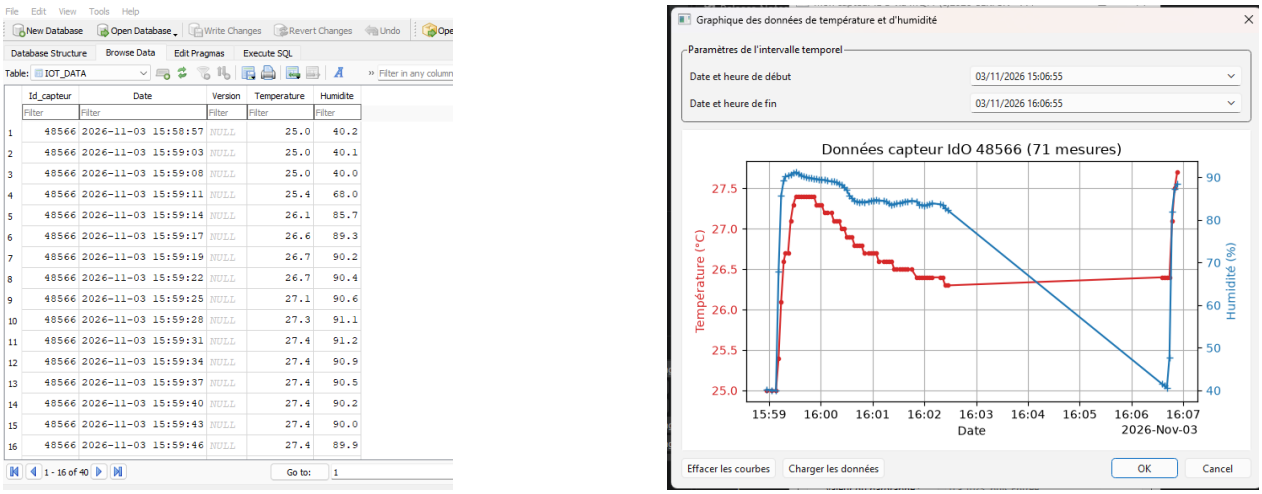


Figure 18 – Table `IOT_DATA` dans DB Browser (à gauche) ; courbes de la dernière heure, avec le moment où j'ai soufflé sur le capteur (à droite).

La fenêtre des courbes lit la base entre deux dates et trace la température et l'humidité avec **matplotlib**, sur deux axes car leurs échelles sont différentes. Sur une heure, 71 mesures ont été archivées, et l'humidité passe de 40 à 90 % après un souffle sur le capteur.

7 Cycle de développement et sécurité

Cours Partie 2 – Cycle de développement et sécurité des objets connectés
Modèle OSI Couches 2 (WPA2) · 6 (TLS) · 7 (authentification, ACL)

7.1 Développement

Le travail a été suivi avec **Git et GitHub** : un dossier par partie (`SIMULIDE`, `TEMPHUMIX`, `IHM`), un commit par étape du TP et un README qui décrit le projet. Les identifiants WiFi et MQTT sont dans des fichiers séparés (`arduino_secrets.h`, `config_mqtt.py`), exclus du dépôt, avec des fichiers modèles `.example`.

7.2 Sécurité par niveau

Le cours présente la sécurité niveau par niveau. Comparé au projet :

Niveau	Ce que recommande le cours	Situation du projet
1 · Perception	Chiffrement, ACL, anti-rejeu, horodatage	Mesures horodatées par le RTC, mais envoyées sans chiffrement
2 · Réseau	Chiffrement, authentification, VPN, certificats	WiFi en WPA2, avec un mot de passe partagé par toute la salle
3 · Application	Authentification, contrôle d'accès	Compte MQTT commun, mot de passe en clair sur le port 1883

Table 8 – Sécurité recommandée par le cours et situation du projet.

> **Question 17 – TP MQTT, question générale 5**
> *Quelles sont les mesures de sécurité entreprises et implémentées dans la dernière version du firmware du routeur ?*

RÉPONSE RutOS propose un pare-feu avec protection contre les attaques (SYN flood, scan de ports), WPA2/WPA3 et l'authentification 802.1X, un mot de passe administrateur unique à changer à la première connexion, l'interface web en HTTPS, l'accès SSH par clé, des VPN et des mises à jour régulières, y compris du modem 5G. Dans les versions récentes (RutOS 7.14), les fichiers système critiques passent en **lecture seule** et des services jugés peu sûrs (POST/GET) ont été retirés. Une faille du modem 5G (« 5Ghoul ») a aussi été corrigée.

Question 18 – TP MQTT, question générale 6

Commentez les logs suivants issus du broker MQTT Mosquitto.

RÉPONSE Les logs montrent le démarrage de **Mosquitto 2.1.2** dans un conteneur Docker (`iot-mosquitto`) ; `1770134524` est un horodatage Unix (3 février 2026).

- **Points positifs** : authentification par mot de passe active (*basic-auth*), ACL actives (*acl-check*), broker lancé sans les droits root ; TLS, WebSockets, bridge et persistance sont disponibles.
- **Faiblesses** : le **TLS n'est pas utilisé**, seul le port 1883 non chiffré est ouvert (en IPv4 et IPv6) ; les fichiers `passwd` et `acl` sont **lisibles par tous** et n'appartiennent pas à l'utilisateur `mosquitto` (il faudrait `chmod 0700` et `chown mosquitto`) ; la règle `TESTS/#` ne contient ni `%u` ni `%c`, donc **chaque utilisateur peut accéder aux topics des autres** ; « Restored 0… » indique un premier démarrage ou une persistance vide.

À RETENIR

Pour sécuriser : MQTT sur **TLS** (port 8883), **un compte par boîtier** avec des ACL du type `pattern readwrite /iot/+_%u`, et des droits corrects sur les fichiers du broker.

8 Difficultés et conclusion

8.1 Difficultés rencontrées

- En simulation, la bonne librairie de l'écran était `LiquidCrystal_AIP31068_I2C.h` et non celle de la documentation.
- Windows repassait tout seul sur le réseau « IMT Nord Europe » et le broker n'était plus joignable.
- Les topics de la carte sont en majuscules alors que les exemples du TP sont en minuscules : MQTT respecte la casse, donc rien n'arrivait au début.
- Le programme d'écriture était déconnecté pendant les saisies au clavier : il a fallu faire tourner la boucle réseau de paho en arrière-plan.
- Les dates au format JJ/MM/AAAA ne se trient pas dans SQLite : il a fallu les convertir en AAAA-MM-JJ.

8.2 Conclusion et perspectives

Ce projet m'a permis de parcourir **tous les niveaux de l'architecture IoT** vus en cours : un objet qui mesure (niveau 1), un réseau WiFi (niveau 2), le protocole MQTT (niveau 3) et une application qui affiche, commande, archive et trace les données (niveau 4). Wireshark m'a permis de voir concrètement le modèle OSI et l'encapsulation, et de constater que **le port 1883 n'est pas sécurisé**. J'ai aussi appris à utiliser un thread pour garder une interface réactive, et à organiser mon travail avec Git.

Pour aller plus loin, on pourrait sécuriser MQTT avec TLS et un compte par boîtier, ajouter un seuil de publication pour ignorer le bruit du capteur, et réaliser le challenge du TP : afficher tous

les capteurs de la salle grâce à un message sérialisé `date|temp|hum`.

Récapitulatif des réponses aux questions

N°	Question du sujet	Section
1	TP partie 1, § 3.1.5 : conclusion sur les différents circuits	3.1
2–4	TP MQTT, § 5.2.4 : autre poste, `TESTS/#`, Retain	5.1
5	TP MQTT, § 6.1.3 : connexion au réseau 5 GHz	4.1
6–8	TP MQTT, § 6.2.1 : mot de passe, taille d'un PUBLISH, Keep Alive	4.3
9–12	TP MQTT, § 6.2.2 : Retain, effacement, QoS 0/1/2, jokers et `$SYS`	5.3
13	Question générale 1 : modèle OSI	4.2
14–16	Questions générales 2 à 4 : routeur RUTX50	4.4
17–18	Questions générales 5 et 6 : firmware, logs Mosquitto	7.2
–	TP IHM : notice de fonctionnement de l'interface	6.2